胡傲东

18085741680 hu.aodong@icloud.com GitHub 个人主页

核心竞争力

前端工程师，专注于复杂交互、跨端桌面界面与高性能渲染，能够将 AI 产品需求抽象为可维护、可扩展的前端架构。以 TypeScript 与 React 18 为核心，擅长组件系统设计、异步数据流管理、Server-Sent Events (SSE) 流式交互与实时预览链路优化，覆盖 Web、WeCom H5 与 Tauri 桌面端。熟悉浏览器渲染机制、事件循环与工程化体系，能够系统解决首屏加载、长列表渲染、内存泄漏和跨端一致性等问题。

React 18 · TypeScript · Tauri / Qt · 前端架构 · 性能调优 · 工程化体系

教育背景

华中科技大学 · 集成电路设计与集成系统（本科） 2022.09 - 2026.06

专业技能

语言与框架
TypeScript、JavaScript (ES6+)、HTML5、CSS3、React 18、Vue 3、Tailwind CSS

前端架构
组件化设计、状态管理、Redux、Zustand、React Query、SWR、可维护前端分层

跨端与交互
Tauri、Qt/QML、WeCom H5、响应式布局、Server-Sent Events (SSE)、实时预览链路

性能优化
浏览器渲染机制、事件循环、首屏加载优化、长列表虚拟化、帧渲染优化、内存泄漏排查

工程化体系
Vite、Webpack、模块化构建、代码规范治理、自动化构建、CI/CD

项目经历

端到端 AI 视频生成系统（跨平台多模态任务平台） 全栈

字节剪映 · 工程营实习生小组长（QML/C++ UI 核心开发） · 2025.11 - 至今

项目定位：将跨平台桌面端打造成支持实时分镜编辑、任务追踪、预览导出一体化的 AI 视频工作台，原生体验下把实时预览帧率从 30fps 提升至 60fps，并在 2 周内完成 MVP
架构取舍：放弃把任务轮询、预览刷新和导出状态堆叠在单页面状态树中的常规做法，采用 Qt/QML 的 MVVM 分层，把进度流、预览流和操作命令拆成独立数据通道，降低状态耦合与组件回流成本
机制设计：通过 QThread + Signal-Slot 隔离渲染线程与计算线程，并为暂停/恢复/取消等破坏性操作定义统一确认与状态回滚语义，避免长任务场景下 UI 假死和多线程竞争
工程结果：帧缓冲复用与增量刷新让多模态实时预览在资源紧张场景下仍保持流畅，配合统一组件规范与 CI/CD 流程，使 5 人协作下的跨端交付节奏稳定可控
技术栈：Qt 6 / QML / C++ · MVVM · FFmpeg 渲染 · Go/Gin 后端对接

AI 智能教学平台 全栈

本科毕业设计 · 独立前端开发（Vite/React/Tauri） · 2025.11 - 至今

项目定位：将教学平台前端重构为覆盖 Tauri 桌面端、企业微信 H5 与移动端的一体化交互层，在 200+ 研究生试点场景中承载学生画像、学习轨迹与 AI 对话的高频操作
架构取舍：未采用页面级状态堆叠与一次性返回答案的简单方案，因为它难以支撑长文本问答、跨端共享与复杂交互；转而构建组件化 SPA，利用 React Query 管理异步状态与缓存，并统一桌面端与 Web 端的数据流语义
机制设计：抽象学生画像可视化、学习轨迹热力图与 AI 对话框等核心组件，封装统一的 Server-Sent Events 流式渲染器，解决长文本答疑中首字延迟高、内容闪烁和滚动抖动的问题
端侧协同：桌面端接入 NPU 加速本地推理，使离线智能答疑成为可用路径；前端通过统一状态层协调本地推理与云端服务回退，兼顾低时延体验与跨端一致性
技术栈：Vite · React 18 · TypeScript · Tauri · React Query · Tailwind CSS

macOS 系统优化工具（SwiftSweep） Apple

个人独立项目 / 开源工具 · 2024.10 - 2024.12

项目定位：为解决 macOS 用户存储空间焦虑与常见清理工具高授权风险，基于 Swift 5.9 与 SwiftUI 独立开发集系统深度清理、相似文件检测与特权隔离于一体的原生维护工具（GUI + CLI），实现了 14+ 核心系统维护功能并成功完成 Apple 官方公证发布
架构与安全设计：摒弃传统清理软件索要全局 root 权限的危险模式，采用 XPC Helper + SMAppService 构建进程间通信隔离层，主应用在沙盒与受限权限下运行，彻底阻断了提权漏洞攻击面；设计了 dry-run 预演、基线白名单与严格的 symlink（符号链接）逃逸检测三重防护体系，实现 100% 零误删系统核心进程数据
算法与性能调优：面对数以万计的相似图片与视频检测导致的内存暴涨与 CPU 满载问题，引入基于 pHash（感知哈希）与 LSH（局部敏感哈希）降维检测算法；利用 Swift Task / Async 机制实现流式处理与内存回收，将重复媒体文件扫描速度提高了 3 倍，内存峰值控制在 200MB 以内
技术栈：Swift 5.9 / SwiftUI · XPC 进程间协议通信 · SMAppService · pHash / LSH 特征匹配算法 · Swift 包管理器 (SPM) · Developer ID 证书体系与 Notarization